

A sub-microsecond relay-BP decoder for the $[[144, 12, 12]]$ gross code: from a fixed-point golden model to FPGA silicon and a 7 nm predictive ASIC

Ruslan Malymon

August 29, 2026

Abstract

We report a complete hardware implementation path for real-time decoding of the $[[144, 12, 12]]$ gross bivariate-bicycle code with relay belief propagation (relay-BP): a fixed-point golden model, a family of banked FPGA cores, silicon campaigns, and a placed-and-routed design on a 7 nm predictive PDK. An 8-bit (Q5.3) min-sum message word reproduces the floating-point logical error rate within statistical error, and a 6×10 leg/iteration schedule is chosen from a budget sweep. The shipped Kria KV260 build (16/48 banking) decodes a circuit-level detector-error-model window in $15.64 \mu\text{s}$ worst case and $0.85 \mu\text{s}$ median with early exit at 133.3 MHz, and is bit-exact against the software golden over 10^6 shots at each of three operating points. The decoder’s convergence flag heralds nearly every logical error: $P(\text{error} \mid \text{converged}) \leq 3 \times 10^{-6}$, so the logical error rate is, to within statistics, the non-convergence rate, and every post-processing fallback we tried was worse than flagging alone. The full-parallel 144/864 geometry decodes in 543 cycles; it places and routes at 76.3% of a Virtex UltraScale+ VU47P at 97.3 MHz ($5.58 \mu\text{s}$) and, through OpenROAD on ASAP7, at 614.6 MHz ($0.88 \mu\text{s}$). Gate-level simulation of the routed 16/48 netlist gives a real-activity power of 0.149 W at 1 GHz, 0.31 μJ per decode window, 93% of it in the clock tree and register clocking. The same simulation exposes an unrepaired hold failure on the latch register file that makes every setup-only frequency figure provisional; we report it rather than hide it. All RTL, goldens, campaign data and a one-command KV260 deployment are open source.

1 Introduction

Quantum low-density parity-check (qLDPC) codes promise fault-tolerant memory at a fraction of the surface code’s qubit overhead [1], and belief-propagation decoders — in particular relay-BP [2] — have been shown, in software, to reach the logical error rates that make such codes useful without an ordered-statistics tail. Whether that translates into a decoder that keeps up with a quantum processor is a hardware question: the decode of one

syndrome window must complete in about a microsecond for superconducting platforms, and in the tens to hundreds of microseconds for ions and neutral atoms, on a budget of area and power that a control system can absorb.

This paper documents one end-to-end answer, built in the open and measured at every step. The contributions, in the order the work was done:

1. A fixed-point relay-BP golden model for the $[[144, 12, 12]]$ gross code that is the RTL specification: an 8-bit Q5.3 message word and a 6×10 schedule, each chosen by a sweep against the floating-point decoder on circuit-level noise (§2).
2. A ladder of synthesizable cores from a sequential FSM to a K -banked, constant-geometry design whose gather/scatter is a fixed permutation network and whose message storage is a latch register file, with a co-simulation harness that drives every variant — and the routed ASIC netlist — with the same golden vectors (§3).
3. Silicon results on a \$300 Kria KV260: $15.64 \mu\text{s}$ worst-case, $0.85 \mu\text{s}$ median, bit-exact over 3×10^6 shots, and the finding that the convergence flag heralds essentially every logical error (§4).
4. The full-parallel geometry on a rented Virtex UltraScale+ and on ASAP7 through OpenROAD: 543 cycles, $0.88 \mu\text{s}$ at 7 nm predictive, with the first power figure derived from gate-level switching activity rather than default toggle rates (§5).
5. The negative results that shaped the design and that a tape-out would have to face: no OSD tail, no feasible NGROUP setting, a streaming core that does not fit, and a latch-clock hold failure that repair cannot buffer away (§6).

Every number in this paper is copied from a dated report in the public repository [3]; Appendix A maps each to its source file. Nothing measured on ASAP7 is called “the chip”: it is a 7 nm *predictive* kit, and the target node is not.

2 The decoder

Code and error models

The target is the gross code, the $[[144, 12, 12]]$ bivariate-bicycle (BB) code of Bravyi et al. [1]: a CSS code over the group algebra of $\mathbb{Z}_\ell \times \mathbb{Z}_m$ with $H_X = [A | B]$, $H_Z = [B^\top | A^\top]$ for two commuting three-term polynomials in the cyclic shifts x, y . The gross code is $\ell = 12$, $m = 6$, $A = x^3 + y + y^2$, $B = y^3 + x + x^2$: $n = 144$, $k = 12$, $d = 12$ (distance cited from [1]; n, k recomputed from $\text{GF}(2)$ ranks), 72 X -checks and 72 Z -checks, every check of weight 6 and every qubit of degree 3. Each error lights three checks, so the syndrome graph is a hypergraph: matching decoders do not apply and belief propagation (BP) is required.

Two detector error models (DEMs) feed the same decoder through a Tanner graph. The *code-capacity* DEM (one perfect round, independent Z error per qubit) has 72 detectors, 12 observables, 144 mechanisms and $144 \times 3 = 432$ Tanner edges; its regular graph is what the unrolled RTL of §3 bakes in, and it is the model of the fixed-point and schedule studies below. The *circuit-level* DEM lays down the depth-7 syndrome-extraction schedule of Bravyi et al. in a memory- X experiment, with the Z -checks measured at round 6 before that round’s X -CNOTs so that no hook error corrupts a stabiliser, and projects the circuit-level depolarising model (CNOT, idle, preparation and measurement faults at rate p) to the Z sector. It is verified edge-for-edge against Stim [4]: on the $[[72, 12, 6]]$ member of the family at rounds $\in \{1, 2, 3\}$ with non-uniform rates every edge probability agrees to below 10^{-9} , and a clean Stim build certifies the schedule is non-disturbing. At rounds $= d = 12$ the gross-code circuit DEM has 936 detectors, 12 observables and 8784 mechanisms, about $13\times$ the detectors and $60\times$ the mechanisms of the code-capacity model.

Relay-BP min-sum

BP alone is degeneracy-limited on qLDPC codes: it oscillates or converges to a valid but wrong-coset solution. The usual remedy, BP followed by ordered-statistics decoding (BP-OSD) [5], adds a data-dependent $\text{GF}(2)$ Gauss–Jordan elimination of unbounded latency. Relay-BP [2] stays inside message passing: several *legs* of normalised min-sum BP, each reseeding a per-variable disordered memory coefficient γ_v and relaying the previous leg’s messages, keeping the lowest-weight syndrome-consistent solution seen. In our software comparison it beat BP-OSD at every p while dropping the elimination entirely, so it is the hardware decoder. The check update is normalised min-sum with $\alpha = 0.875$; plain $\alpha = 1$ over-converged to wrong cosets on 29% of shots at $p = 0.04$ (code capacity), against 2.3% at $\alpha = 0.875$. The (legs, iterations per leg) schedule is fixed, so the worst-case latency is a constant.

Table 1: Message-width sweep, code-capacity gross code: logical error rate of the fixed-point relay-BP decoder as a ratio to the **f64** decoder (1.00 = identical), 20 000 shots per point, seed 2024. The **f64** rates are 1.00×10^{-4} , 2.00×10^{-3} , 1.27×10^{-2} and 4.35×10^{-2} at $p = 0.02, 0.03, 0.04, 0.05$. Every cell is within the Monte-Carlo 95% CI of the **f64** rate. Q5.3 is the word carried to RTL.

(W, F)	range / step	$p=0.02$	0.03	0.04	0.05
(6, 2)	$\pm 7.75 / 0.25$	2.00	1.20	1.06	0.97
(7, 3)	$\pm 7.88 / 0.125$	2.50	1.18	1.04	0.99
(8, 3) Q5.3	$\pm 15.9 / 0.125$	1.00	0.97	0.95	0.96
(8, 4)	$\pm 7.94 / 0.0625$	1.50	1.18	0.98	0.99
(10, 4)	$\pm 31.9 / 0.0625$	1.00	0.97	0.97	0.98
(12, 6)	$\pm 31.9 / 0.016$	1.00	0.95	0.99	0.99

Fixed-point word

The bit-accurate golden model **FixedRelayBp** quantises the arithmetic and is, by construction, the RTL specification. A value is stored as signed $\text{round}(x \cdot 2^F)$ saturated to $\pm(2^{W-1} - 1)$, with W message bits and F fractional bits. Since $\alpha = 7/8$, normalisation is $\text{mag} - (\text{mag} \gg 3)$ and the whole check update is $\text{compare}/\text{min}/\text{sign}$ with no multiplier. The only multiply is the relay blend $(1 - \gamma_v) \cdot \text{computed} + \gamma_v \cdot m_{\text{old}}$ with γ_v a per-variable ROM constant, i.e. a fixed-coefficient multiply. The blend truncates by arithmetic shift (floor), the cheapest RTL rounding, and the golden adopts the same rule so software and silicon agree bit for bit.

Table 1 sweeps (W, F) against the **f64** decoder on the code-capacity model, 20 000 shots per point. Every cell is within the Monte-Carlo CI of **f64**, but (8, 3), Q5.3 with range ± 15.9 and step 0.125, tracks it most tightly and is the only 8-bit word that reproduces the **f64** rate exactly at the $p = 0.02$ floor (1.0×10^{-4}). It beats the finer (8, 4) there on dynamic range: at low p the priors are largest ($\lambda(0.02) \approx 3.9$, $\lambda(0.01) \approx 4.6$) and messages accumulate, so the integer range matters more than a fractional bit. Below 8 bits, (6, 2) and (7, 3) bias high by $2\text{--}2.5\times$ at the floor, where accuracy matters most: 8 bits is the floor, not a margin. The truncating blend acts as mild extra damping, which is why Q5.3 sits slightly below **f64** ($0.95\text{--}0.97\times$) at higher p . At Q5.3 the message RAM (432 edges $\times 2 \times 8$ b) is 864 B, about $8\times$ smaller than at **f64**.

Leg/iteration schedule

In the unrolled RTL a worst-case decode takes $3 \cdot \text{legs} \cdot \text{iters} + 1$ cycles, so the schedule is the dominant latency term: the software default 4×25 costs 301 cycles, $3.16 \mu\text{s}$ at the 95.2 MHz of the first KV260 build. We swept (legs, iters) at the Q5.3 word on the code-capacity model, 80 000 shots per point, against the full 4×25 decoder (Fig. 1). First, at a fixed sweep budget the split matters more than the total: the 60-sweep schedules 6×10 , 5×12 , 4×15 and 3×20 all cost 181 cycles ($1.90 \mu\text{s}$), but

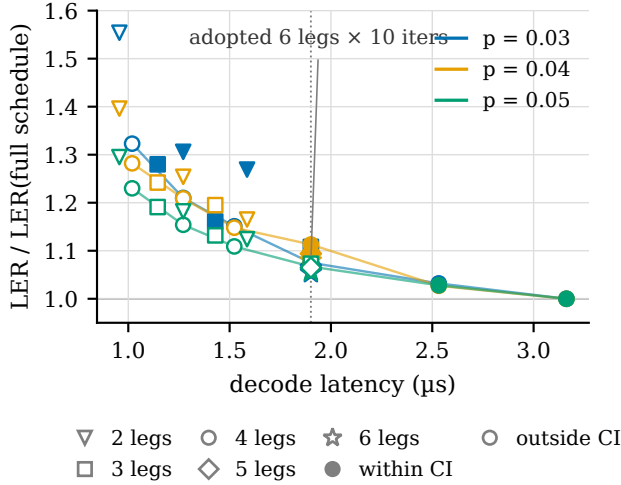


Figure 1: Leg/iteration budget sweep at the Q5.3 word (code-capacity gross code, 80 000 shots per point). Each point is one (legs, iters) schedule: worst-case RTL latency at 95.2 MHz ($3 \cdot \text{legs} \cdot \text{iters} + 1$ cycles) against logical error rate as a ratio to the full 4×25 schedule, at $p \in \{0.03, 0.04, 0.05\}$. Filled markers are within the Monte-Carlo CI of the baseline. The 181-cycle schedules differ only in the split; 6×10 is the one adopted.

at the discriminating $p = 0.05$ only 6×10 stays within CI of the baseline ($1.057\times$, against $1.065\times$, $1.067\times$ and $1.073\times$). Relay-BP’s strength is leg-to-leg disorder diversity, and iterations within a leg have diminishing returns once BP has settled, so cycles are best spent on many short legs. Second, the schedule alone cannot reach sub-microsecond latency: the only sub- μs schedule, 2×15 (91 cycles, $0.96 \mu\text{s}$), costs $1.295\times$ LER and is outside CI at every p . Adopting 6×10 (a six-leg γ ROM, legs 0–3 keeping their seeds) and narrowing the blend accumulator from 32 to 16 bits (bit-exact, since $|\text{blend}| \lesssim 5600$) cut the worst case from 301 to 181 cycles; the KV260 build closed at 96.0 MHz, $1.89 \mu\text{s}$, $1.68\times$ faster than 4×25 at $1.06\times$ LER; the gain is almost entirely the cycle cut.

Circuit-level decoding

The same fixed-point decoder handles gate-noise syndromes. On the depth-7 circuit-level DEM at rounds = 6 with the 4×25 schedule, plain Q5.3 relay-BP tracks the float decoder and its LER falls with p below a per-cycle threshold of about 0.3%: 4×10^{-3} at $p = 0.002$, 2×10^{-2} at $p = 0.003$, 6.5×10^{-2} at $p = 0.004$. This agrees with the software study, where the per-cycle rate of the $d = 12$ code crossed below the $d = 6$ member at $p \approx 0.0025$ – 0.003 ; Bravyi et al. report about 0.7% for the same code, and since the DEM is Stim-exact the remaining 2–2.5 $\times$ is decoder strength, not the model. The circuit-level graph is irregular: at rounds = 1 it has 864 variables, 144 checks and 2952 edges with maximum check degree 25, against

a uniform 6 at code capacity. The parametric sequential RTL decoder (§3) decodes 40 sampled DEM shots over this graph bit-identically to the golden with no RTL change, only a regenerated Tanner header.

No OSD tail (negative result)

Relay-BP occasionally leaves a hard decision with $H\hat{e} \neq s$. We built the classic fix, an OSD tail on the fixed-point posterior LLRs (`FixedRelayBpOsd`), as a slow path on the processor side gated by the RTL’s `valid_flag`, and measured whether it earns its place. At code capacity (20 000 shots) OSD-0 is LER-neutral at every p within CI while the tail rate grows from 0.15% to 8.6% over $p = 0.03 \rightarrow 0.06$: the residual failures are mostly uncorrectable (weight above $d/2$), so a valid OSD decode is a near coin-flip between cosets. At circuit level (rounds = 6, 3000 shots) OSD-0 is worse than no tail in both fixed and float arithmetic: at $p = 0.002$ the plain fixed/float LER $4.0 \times 10^{-3}/3.3 \times 10^{-3}$ rises to $6.7 \times 10^{-3}/5.7 \times 10^{-3}$, and at $p = 0.003$ $2.0 \times 10^{-2}/1.9 \times 10^{-2}$ rises to $2.7 \times 10^{-2}/2.8 \times 10^{-2}$; replacing an invalid guess by a valid but often wrong-coset decode loses more shots than it rescues. Order 4 is about break-even, and the beneficial regime is order 12 ($1.7 \times 10^{-3}/1.3 \times 10^{-3}$ and $1.3 \times 10^{-2}/1.5 \times 10^{-2}$), i.e. $2^{12} = 4096$ reliability-ordered GF(2) re-solves per failed shot: exactly the data-dependent, variable-latency work the hardware track set out to avoid. Fixed Q5.3 tracks float at every order, so the word is not the limiter; the order is. The decoder therefore ships as pure relay-BP with no OSD tail.

3 Hardware architecture

The decoder RTL was reached by a ladder of six synthesised variants, each bit-exact against the same fixed-point golden model (§2) and each measured out-of-context (OOC) on the target part before the next step was chosen. Table 2 summarises the ladder; the rest of this section explains what each step learned. Two graph sizes appear: M2–M5 decode the code-capacity Tanner graph of $[[144, 12, 12]]$ (72 checks, 144 variables, 432 edges), while M6 onward decode the circuit-level detector-error-model graph ($C = 144$ checks, $N = 864$ variables, $E = 2952$ edges, maximum check degree 25).

M2: sequential FSM

The first synthesisable decoder time-multiplexes the relay-BP update into a clocked FSM that touches one check (≤ 6 edges) or one variable (≤ 3 edges) per cycle, so combinational depth is bounded and independent of graph size. One iteration is a 72-cycle check pass, a 144-cycle variable pass (the only multiply: the memory blend $(1 - \gamma) \cdot \text{computed} + \gamma \cdot m_{\text{old}}$) and a 72-cycle satisfaction pass; the 4×25 schedule plus a 144-cycle emit gives a worst case of 28 944 cycles per decode.

Table 2: Architecture ladder. Cycles are the full-schedule (no early exit) worst case; fabric is the post-synthesis LUT count and share of the Xilinx Kria KV260 (XCK26, 117 120 LUTs). Clocks are OOC for M2–M5 and M9c, the timing-met board clock for M6–M8.

Variant	Graph	Cycles	Fabric (LUT)	Clock / latency	Verdict
M2 sequential FSM	code cap.	28 944	24 119 (21 %)	67.8 MHz / 427 μ s	cursor mux is the wall
M4 spatial unroll	code cap.	301	94 194 (80 %)	95.2 MHz / 3.16 μ s	mux gone; arithmetic depth
M5 SAT-overlap (6×10)	code cap.	122	93 487 (79.8 %)	95.9 MHz / 1.272 μ s	1.48 $\times$ free; route-bound
M6 edge-serial BRAM	circuit	672 000	8 509 (7.3 %)	100 MHz / 6.72 ms	unroll fails to synthesise
M7 banked 12/36	circuit	2 460	88.0 k (75 %)	75 MHz / 32.8 μ s	on silicon, 40/40
M8 banked 16/48	circuit	2 085	102.8 k (88 %)	133.332 MHz / 15.64 μ s	on silicon, 40/40
M9c full-parallel gather	circuit, streaming	4 475	189 740 (162 %)	177.7 MHz OOC	no fit on one KV260

M3: out-of-context synthesis verdict

The FSM fits both parts with no DSP and no BRAM, but at 28.3 MHz on the Zybo (1023 μ s) and 67.8 MHz on the KV260 (427 μ s) it is two to three orders of magnitude off real time. The binding path is not the multiply—the per-variable γ constants fold into LUTs and DSP usage is zero—but the runtime check cursor: reading `BP_CHECK_OFF[idx]` then the edge list synthesises to a 72-way select in front of the min-sum whose result fans out through a 432-way write decode (net fan-outs of 586 and 626; 44 logic levels and 74 % routing delay on the Zybo). The lesson that shaped everything after it: on a constant-degree graph, a runtime index into a message array is a mux tree the size of the array.

M4: spatial unroll

Wiring each check’s six edges and each variable’s three edges as compile-time constants deletes the cursor entirely and lets a whole BP layer evaluate in one cycle: 301 cycles at 4×25 , 96 $\times$ fewer than M2. On the KV260 the unrolled datapath needs 94 194 LUTs (80 %) and closes at 95.2 MHz, 3.16 μ s—135 $\times$ faster than M3; on the Zybo it is 172.6 % of the part and does not fit. The new critical path is real arithmetic (the variable-update blend, 25 logic levels through five CARRY8 chains) plus routing congestion at 80 % utilisation. A synthesis caveat is worth recording: with every index constant, Vivado constant-propagated the 100-iteration message recurrence to a bogus fixed point and deleted the datapath, leaving a 158-flop shell that “closed timing” at ~ 485 MHz; the registers had to be anchored with `dont_touch`, and post-synthesis flop counts are now checked against elaboration on every variant.

M5: the two microarchitectural levers, measured

Adopting the LER-optimal 6×10 schedule (§2) cut the cycle count to 181 and latency to 1.89 μ s at 96.0 MHz. Two further levers were then tried on the KV260. *SAT-overlap* folds the syndrome check into the following check pass, since the two read disjoint registers: 2 cycles per iteration instead of 3, 122 cycles, 1.272 μ s at 95.9 MHz on 93 487

LUTs—a 1.48 $\times$ win at unchanged clock and area. *Pipelining the min-sum* is the measured negative result: splitting the check pass raised the clock only 96.0 $\rightarrow$ 111.3 MHz (1.16 $\times$) against a 1.33 $\times$ cycle penalty, for 2.165 μ s, a 15 % regression, while pushing utilisation to 85.5 % and worsening congestion. The wall at this point is routing at ~ 80 % utilisation, not logic depth; removing work helps, adding registers does not.

M6: the circuit-level graph does not unroll

On the circuit-level graph (degree up to 25) neither the full unroll nor a CHK12/VAR24 partial unroll synthesises on the KV260: Vivado’s cross-boundary area optimisation ran out of memory at ~ 43 GB or stalled without progress. Only BRAM-resident edge-serial cores fit (5.5–7.3 % LUT), and the two-edges-per-cycle variant gives 672 000 cycles, 6.72 ms at 100 MHz on silicon.

M7: K -banked relay-BP

The design space was a trilemma, all corners measured: the full spatial unroll (144 check + 864 variable submodules) is 453 k LUTs, 386 % of the KV260; a partial unroll with a runtime group gather from flop arrays stalls area optimisation on $O(E)$ operand muxes—the M3 cursor-mux wall again; the banked LUTRAM store synthesises at 88 k LUTs (75 %). In the banked core, W checks and V variables update per cycle. Messages live in hundreds of small distributed-RAM banks in *check-major* order: edge e is stored at half-bank $(\text{slot}(\text{check}(e)) \cdot 25 + \text{pos}(e)) \cdot 2 + \beta(e)$ in the row of its check group, so check-phase accesses are hardwired (broadcast row, 2:1 β muxes) and the only scattered access is the variable phase. An offline solve in the graph emitter (slot assignment, cap-2 variable grouping, β assignment; König edge colouring as the existence fallback) guarantees at most one write per $\mathbf{m_cm}$ half-bank and two reads per $\mathbf{e_cm}$ bank per cycle. Three stores result— $\mathbf{m_cm}$ (variable $\rightarrow$ check, check-major, β -split), $\mathbf{e_cm}$ (check $\rightarrow$ variable, two read ports) and $\mathbf{m_vm}$ (a variable-major shadow that makes the old-message read mux-free)—all distributed RAM, zero BRAM tiles, about 7 k LUTs for the message fabric. Because banking never touches the logical in-check edge order (the min-sum tie-breaks) or the wrap-add order, the (W, V) geom-

etry is a pure scheduling change: the same golden vectors pass at 8/24, 12/36 and 16/48. Getting it through synthesis took a hunt: the flat top churned for ~ 8 h into a 386 k-LUT netlist at 24 MHz; the culprit was a γ table indexed by a runtime 32-bit leg counter, expanded at each of ~ 864 gather sites into a 5184:1 ROM mux plus 32-bit index arithmetic. Constant-folding over the leg (a 6:1 mux) brought it to 71 k LUTs, and replacing serial minimum and XOR reductions with tournament trees lifted the clock 26 $\rightarrow$ 93 MHz, every step re-proved bit-exact. At 12/36 the core is 2 460 cycles; the board build closes at 75 MHz, and on the KV260 the full schedule is 32.8 μ s fixed while early exit gives a mean of 2.5 μ s (median 1.9, p99 9.3), 40/40 shots bit-exact in both modes—205 $\times$ M6 worst case.

M8: the KV260 squeeze

M7’s unregistered bank-read $\rightarrow$ gather $\rightarrow$ min-sum path is broken by a register plane on the gather outputs and a third `check_minsum` stage (+3 cycles per iteration), and the 16/48 geometry—originally rejected on a DSP count that was 9 $\times$ inflated by a counter matching DSP sub-primitives—turns out to time better than 12/36. Cycle counts become 3 750 / 2 640 / 2 085 at 8/24 / 12/36 / 16/48; OOC 16/48 is 102.8 k LUTs (88%), 164 DSPs (13%), 177.7 MHz. The board build meets timing at 133.332 MHz (the PS clock grid), and on silicon the full schedule is 15.64 μ s, early exit mean 1.15 μ s, median 0.85 μ s, p99 4.4 μ s, 40/40 bit-exact. LUT utilisation at 88% is the ceiling for wider geometries on this part.

Constant-geometry gather and scatter

The fully parallel streaming core (M9b, §4) exposed the mux wall in its purest form. Its first version indexed register arrays with ROM-held values; a serial synthesis on a 123 GB host placed it at 2 232 451 LUTs, 1906% of the KV260, entirely from three runtime-indexed crossbars (F7/F8 muxes at 238%/231%), while the non-crossbar core fit. The gather is a large fixed permutation, so it was replaced by ROM-configured Beneš networks (one size-1024 network for the `m_cm` write, two size-512 networks per port for the dual-port `e_cm` read), with control pipelined in lockstep with data: 239 750 LUTs, 9.3 $\times$ fewer, 177.7 MHz, latency 2 206 $\rightarrow$ 2 810 cycles because the fabrics sit inside the non-overlapped iteration loop. A memory-based serial gather (P BRAM banks read over $\lceil N/P \rceil$ steps) was designed and area-probed and does not help: an edge cannot occupy the same bank position in both its write group and its read group, so a per-tap residual select remains, and that select is $O(N \cdot \text{STEPS})$ —the same order as the crossbar (about 255 k LUTs estimated, worse than Beneš); only a fully serial $P \leq 2$ gather is mux-free and it costs hundreds of microseconds. Two right-sizings followed. The `e_cm` read-address network routed no runtime data (both its input and its control are

per-group constants), so it became a sync-read BRAM ROM: $-32\,819$ LUTs and -54 BRAM tiles, which also exposed that the Beneš core had been over budget on BRAM tiles (151.7%) as well as LUTs. The remaining power-of-two-padded fabrics were replaced by arbitrary-size Waksman networks [6, 7] sized to the real dimensions ($N = 800$ and $N = 400$ for the 288-tap partial injections), switch-optimal at $\lceil N \log_2 N \rceil - N + 1$ switches (`m_cm` 9728 $\rightarrow$ 6977, `e_cm` 4352 $\rightarrow$ 3089 per port): a further $-17\,191$ LUTs. The campaign closes at 189 740 LUTs (162.0%) and 163 BRAM tiles (113.2%), bit-exact 40/40 at both bankings, latency 4 475 / 2 810 cycles. The floor is structural: the two runtime-data fabrics that cannot become ROMs are $\approx 108\%$ of the KV260 and the non-fabric decoder is $\approx 68\%$, so no right-sizing brings the sum under 100%; a fully parallel gather for this graph needs a larger device. The same fixed fabrics (`bp_asw.sv`, `bp_benes.sv`) are the ones the ASIC architecture carries, with bank permutations loaded at boot rather than baked, so one die serves a code family while the fabric geometry stays fixed.

ASIC register-file restructuring

On an ASIC the dominant storage shape is the 8b $\times$ 9 and 8b $\times$ 18 message array, more than 1 200 instances, which the architecture maps to flop or latch register files rather than SRAM macros. A flat-flop elaboration of the M8 core synthesised to 5.28 mm² on sky130hd (OpenROAD flow [8]) but failed global routing on met2 at both 30% and 20% utilisation. Q7-08 restructured the three cell types according to their access patterns: the 164 `m_vm` cells share both cursors across the lanes of a variable slot and consolidate into 48 wide byte-masked arrays (`VAR_DEG` $\times$ 8b $\times$ 18 rows, one row decoder per slot); the ~ 740 `m_cm` cells have a per-cell write row and the ~ 370 `e_cm` cells two fabric-driven read rows, so both become transparent-low latch arrays—consolidating them would rebuild the runtime-index mux that M7 removed. Latch storage measured 15.1 μ m²/bit against 38.4 for flops (2.54 $\times$ denser), and the restructured core is 4.54 mm² (-14%) with 43 k clock sinks instead of 98.8 k; the latch write is safe because `m_cm/e_cm` reads and writes occur in different FSM states. Routability, however, did not follow: at 20% utilisation the datapath alone is only marginally better (met2 congestion -30% , overflow -12% , met2 usage $\sim 75\%$ either way), post-CTS both cores sit at 78–82% met2, and the latch-dense clock tree livelocks global routing on clock non-default rules for over 20 h. The identical netlist (43 025 flops, 55 703 D-latches) on the 7 nm predictive ASAP7 platform [9] at 45% utilisation global-routes with zero overflow on every layer M2–M7, no clock-rule disables, and completes detailed routing to a 0.163 mm² die with 207 residual DRC violations, mostly predictive-PDK end-of-line and cut-spacing artefacts (155 at 35%). The routing wall is the 130 nm node’s track density, not the design; timing and power are

not quotable from these runs because `repair_timing` was skipped, and are taken up in §5.

Co-simulation harness

Every variant in Table 2 is driven by the same Verilator testbench and the same golden vectors emitted by the Rust fixed-point model, so a change is accepted only if the chosen error, observable flips and validity flag are bit-identical to software. The code-capacity cores pass 65/65 decodes (the empty syndrome, every single-variable error and 40 random low-weight syndromes); the circuit-level cores pass 40 circuit shots at $p = 0.003$ in both exit modes, and the banked core passes them at all three geometries, in both the flop and the latch/register-file storage style, with identical worst-case cycle counts—the (W, V) -invariance gate. Because 40 shots at $p = 0.003$ never exercised high-weight syndromes, a separate gate decodes 2 000 shots at $p = 0.007$ and requires 2 000/2 000 agreement; the same gate reproduced off-board a campaign divergence that turned out to be a prior mismatch between bitstream and golden rather than an RTL defect. The gates run on macOS and on the EPYC CI host, through the AXI wrapper, and on silicon, so the number that reaches the board has already matched the golden at every layer below it.

4 Results on FPGA silicon

All silicon results in this section are from one board, an AMD Kria KV260 (Zynq UltraScale+, xck26), running the banked relay-BP core of Section 3 in its 16/48 configuration, i.e. 16 check groups and 48 variable groups updated per cycle. The board is the \$300-class part the appliance ships on; nothing here is projected from a larger device.

The shipped build

The 16/48 build closes timing at a PS-granted clock of 133.332 MHz (`TIMING_MET`, WNS 0.000 ns); the two earlier attempts at 150 MHz missed by -0.238 ns (default strategy) and -0.147 ns (`Performance_Explore`). The out-of-context probe at a 5 ns target used 102.8 k LUTs (88 % of the part) and 164 DSPs (13 %). The full best-kept schedule of 6 legs $\times$ 10 iterations takes a fixed 2 085 cycles, so the worst-case decode latency is 15.64 μ s, with min = p50 = mean = p99 = max. With early exit enabled (stop at the first syndrome-satisfying decision), the same silicon on the same 40 circuit-level shots at $p = 0.003$ gives min 79, p50 113, mean 153, p99 = max 589 cycles, i.e. 0.59 μ s minimum, 0.85 μ s median, 1.15 μ s mean and 4.4 μ s worst on that sample. The median decode is therefore sub-microsecond on the shipped board; the worst case is not, and early exit does not change the worst case. Both modes are bit-exact against the software golden on all 40

shots (40/40), and the wrapper identifies itself with ID-CODE 0x4250_0003. For scale, the same board’s worst case went 6.72 ms (edge-serial BRAM core, M6) $\rightarrow$ 32.8 μ s (first banked core, M7) $\rightarrow$ 15.64 μ s (this build): $2.1\times$ over M7 and $430\times$ over M6.

This is the configuration published as `appliance-v1`: the released bitstream, its `.hwh`, the golden vectors and a driver, installed by a script that requires the 40/40 self-test to pass before declaring success (Section 7).

Batched path and the matched-prior campaign

The AXI4-Lite register interface is a per-decode round trip through the processing system, and the processor dominates it: the per-word Python/MMIO runner reaches 3 018 experiments/s at the full schedule and 3 392/s with early exit (331 and 295 μ s per experiment, roughly 94 % harness overhead). An AXI4-Stream batch shell around the same core, fed by AXI-DMA from PS DDR, streams a whole batch of independent syndrome $\rightarrow$ result experiments through one transfer; the per-experiment output is a 32-bit status word carrying the 12 observable flips, the `valid_flag` (bit 19) and the latency in cycles. On silicon (batched overlay at 100 MHz) this gives 43 380 experiments/s at the full schedule (23.1 μ s each, $14.4\times$, now bound by the 2 085-cycle decode itself) and 553 000/s with early exit (1.81 μ s each, $163\times$ the per-word early-exit rate), clearing the $\geq 100\times$ target. Batching removes the harness, after which the decode mode alone sets the ceiling. The batched path matches the golden 40/40 at batch sizes 1, 2, 4, 40 and 20 000, and the same core in the AXI4-Lite overlay also passes 40/40, which cross-checks the core independently of the DMA path.

The batched path made a large on-silicon logical-error-rate (LER) campaign affordable. Circuit-level shots (one round, seed 2024) were sampled at three physical error rates and decoded both by the software fixed-point golden and by the board, 10^6 shots per point, each point against a bitstream whose header bakes that point’s priors. Table 3 gives the result: the RTL LER equals the software LER at every point and the per-shot divergence count is 0 in 3×10^6 shots. The silicon is bit-exact to the golden, not merely within its confidence interval, at 23.09 μ s per shot at the full schedule.

A methodological finding. The first run of this campaign reported 0, 7 067 and 30 703 divergences at the three points, with the RTL LER 7–13 % worse than software at $p \geq 0.005$ and the gap growing with p , and it was initially written up as a synthesis-versus-simulation bug in the banked core. It was not. The shipped overlay’s header had been generated at $p = 0.003$, so the RTL baked the per-variable priors $\lambda(p=0.003)$; the campaign’s golden was built from a detector-error model at the sampling rate, so it decoded with $\lambda(p=0.005)$ and

Table 3: On-silicon campaign, KV260, banked 16/48, circuit-level noise, one round, 10^6 shots at every point, one matched-prior bitstream per point. LER is the logical error rate (software golden and RTL are identical to the digit); mism. counts shots whose RTL observable differs from the golden’s; r is the non-convergence rate $P(\text{valid_flag} = 0)$ with its Wald 95 % interval.

p	mism.	LER	r	95 % CI
0.003	0	8.3200×10^{-4}	1.1680×10^{-3}	$\pm 6.70 \times 10^{-5}$
0.005	0	7.0520×10^{-3}	8.4730×10^{-3}	$\pm 1.80 \times 10^{-4}$
0.007	0	2.8774×10^{-2}	3.2591×10^{-2}	$\pm 3.48 \times 10^{-4}$

$\lambda(p=0.007)$. Same Tanner graph, different priors, hence a different message trajectory on exactly those syndromes hard enough for the trajectory to decide the outcome—which is why the pattern was deterministic, zero at $p = 0.003$, monotone in $|p - 0.003|$, and identical in RTL simulation, functional simulation and silicon. Two reproductions confirmed it: the 24 diverging shots captured from the board re-decode 24/24 to the silicon’s answer in software at $p = 0.003$ (6/24 at 0.005, 0/24 at 0.007), and off-board co-simulation of 2000 shots sampled at $p = 0.007$ against the $p = 0.003$ header mismatches 210/2000 with a golden at $\lambda(0.007)$ and 0/2000 with a golden at $\lambda(0.003)$. The general lesson is that a bit-exactness claim for a decoder with baked-in priors is only meaningful when the golden’s priors are the bitstream’s; the vector emitters now take the decoder’s p as an explicit argument and stamp it into the file header, and a 2000-shot high-weight gate at $p = 0.007$ joined the regression suite because every earlier simulation gate had driven 40 shots at $p = 0.003$ and never exercised heavy syndromes.

Non-convergence: `valid_flag` as a herald

Relay-BP either finds a syndrome-satisfying decision within its budget or emits its best-kept decision with `valid_flag` = 0. The non-convergence rate $r(p)$, measured in software over the same 10^6 shots per point (the LER column reproduces the silicon campaign exactly: same sampler, seed and decoder), is the last column of Table 3; the median iteration to first validity is 2, 4 and 6 at the three points, which is what makes early exit worth 163 $\times$. Splitting the logical errors by the flag gives the central result of this section (Figure 2). At $p = 0.003$, all 832 logical errors in 10^6 shots were shots the decoder had already flagged; the converged population, 998 832 shots, contained zero logical errors, which bounds $P(\text{err} \mid \text{valid}=1) \leq 3.0 \times 10^{-6}$ at 95 % confidence (rule of three). At $p = 0.005$ and 0.007 the conditional error rate of a converged decode is 1.916×10^{-5} and 1.178×10^{-4} . The attributable fraction $A(p)$, the share of logical errors that carry `valid_flag` = 0, is 1.0000 (observed; ≥ 0.9964 at 95 %), 0.9973 and 0.9960. In words: the decoder’s LER is essentially its non-convergence rate, and a converged decision is essentially never wrong. A converged decode

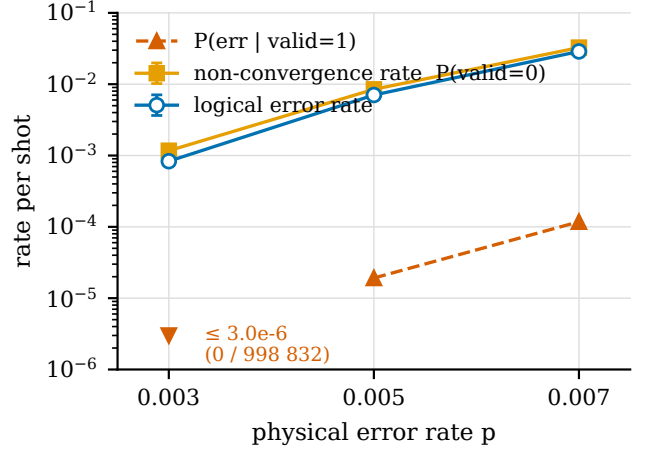


Figure 2: Logical error rate and non-convergence rate $r(p) = P(\text{valid_flag} = 0)$ per physical error rate p , block path, 10^6 shots per point. The two curves nearly coincide because almost every logical error is a non-converged shot: the attributable fraction $A(p)$ is 1.0000 / 0.9973 / 0.9960 at $p = 0.003$ / 0.005 / 0.007, and the error rate conditioned on convergence is $\leq 3.0 \times 10^{-6}$ (zero observed in 998 832 converged shots), 1.9×10^{-5} and 1.2×10^{-4} .

is two to three orders of magnitude more reliable than the raw LER at every point, so a consumer that can discard or escalate flagged shots obtains a post-selected LER of that order for free.

Because $A \approx 1$, an oracle fallback acting only on flagged shots could in principle remove the whole LER, so a fallback cannot be dismissed by arithmetic; the decision rule, fixed before the data were seen, required a candidate to show a significant paired rescue on the non-converged subset and fit the latency budget. We retained 1000 non-converged shots per point as a fixed corpus and re-decoded each with five ordered-statistics-decoding (OSD) candidates: OSD-0, OSD-2, OSD-4, and residual-restricted OSD-2 and OSD-4 (1, 4, 16, 4 and 16 solves per shot). Every candidate is worse than the baseline at every point, and significantly so under a paired McNemar test (every $\chi^2 > 3.84$; the best row, residual OSD-4 at $p = 0.003$, rescues 116 shots and breaks 169, $\chi^2 = 9.49$). Propagated as $\Delta \text{LER} = r(p)[P(\text{err} \mid v=0) - P(\text{err} \mid v=0, \text{fallback})]$, the candidates raise LER by between +5.4 % and +39.9 %; OSD-0 turns a 71 % conditional error rate into 99.6 %. The reason is that the BP posteriors that order OSD’s reliable basis are exactly the ones that just failed to converge: forcing $H\hat{e} = s$ lands the solution in the wrong logical class. The candidates also fail on latency: single-threaded on an EPYC 8124P, a processor-side tail costs 1460–1644 μs per shot it fires on (including the software re-decode the flag-only RTL export forces), against a 1 μs -per-round budget; a tail that fires on 0.12 % of shots still sets the worst case. The chosen policy is

therefore *do-nothing-but-flag*: emit the best-kept decision, export `valid_flag`, no RTL change. Its LER impact is zero by construction. This is reached for the opposite reason to the one the rule’s first branch anticipated (“non-convergence is negligible”): non-convergence is nearly the entire error budget, and nothing affordable recovers any of it. The two directions that could—more relay legs on a flagged shot, or escalation to a host for non-real-time use—are outside this work.

Finally, the flag the policy rests on is the silicon’s flag. On the matched-prior $p = 0.005$ overlay, 10^5 fresh shots gave identical LER in software and hardware (7.1200×10^{-3} , 712 errors each), 0/100 000 observable divergences, and 857 non-converged shots (0.857%) on both sides with `valid_mismatch` = 0. This is one overlay, one operating point and the block path only; at the other two points the licence remains the observable-level bit-exactness of Table 3 plus the off-board co-simulation gate, which compares the flag.

Power

The Kria K26 SOM exposes a single INA260 monitor on its 5 V input rail, so the only observable is SOM-total power (PS, PL, DDR and fan); there is no PL-domain shunt. The measurement is therefore a delta between the PL programmed-but-idle and the PL under a sustained decode stream, with a processor-poll-only control (MMIO reads, no decode) to bound the share of the delta owed to the host loop. On the shipped 16/48 overlay at 133.332 MHz, idle SOM power is 3.25 W ($5.07 \text{ V} \times 0.64 \text{ A}$); a full-schedule decode stream through the AXI4-Lite runner lifts it to 3.50 W (+249 mW, 2 887 decodes/s, PL duty 4.5%), early exit to 3.46 W (+210 mW, duty 0.4%), and the poll-only control alone to 3.45 W (+200 mW). Two runs reproduce to $\pm 5 \text{ mW}$. The host loop is thus about 80% of the load delta; the decode datapath adds only $\sim 48 \text{ mW}$ (full) and $\sim 10 \text{ mW}$ (early exit) at the SOM level. Energy per decode follows in two framings. As shipped through the per-word harness, $\Delta P \cdot$ wall-per-decode is $\approx 86 \mu\text{J}$ (full schedule) and $\approx 67 \mu\text{J}$ (early exit); this is real but harness-bound. Subtracting the control gives a PL-core dynamic energy of $\lesssim 17 \mu\text{J}$ (full) and $\lesssim 3 \mu\text{J}$ (early exit), equivalent to about 1 W of active PL power over the $15.64 \mu\text{s}$ busy window. These are order-of-magnitude upper bounds: they are a small difference of two $\sim 200 \text{ mW}$ numbers, and the control’s read mix does not exactly match the decode loop’s. The batched DMA path raises the PL duty well above 4.5% and would turn the bound into a tight number; that re-measurement has not been done.

5 Scaling to the full-parallel geometry and a predictive ASIC

The shipped 16/48 core (§4) decodes a worst-case window in 2085 cycles, $15.64 \mu\text{s}$ at the KV260’s 133.3 MHz. Latency is cycles divided by clock, so the two questions this section answers are how far the cycle count can be pushed down by widening the core, and how far the clock can be pushed up by leaving the FPGA. Both were measured rather than modelled, and two of the three answers came out worse than the plan assumed. We report them in the order they were obtained.

The banking family is the only implementable one

Two architectures span the design space between the banked core and a fully unrolled one. `bp_relay_unroll_pipe` stamps $1/NGROUP$ of the check and variable slots and time-multiplexes each across $NGROUP$ groups per BP phase. Its cycle count is exactly affine, $\text{cycles}(NGROUP) = 122 \cdot NGROUP + 240$, and the co-simulation is 40/40 bit-identical to the golden at every setting measured. The knob nevertheless has no feasible setting. Five values were synthesised out-of-context on the KV260 part at a 5.0 ns target: every one is over the device budget *and* slower than the banked core. The cheapest member, $NGROUP = 144$ with one check slot and six variable slots stamped, still needs 703 698 LUTs (601% of the device) at 17.5 MHz; the fastest, $NGROUP = 16$, reaches $132.8 \mu\text{s}$ at 1474% of the device, $8.5\times$ slower than the shipped core. $F_{\max}$ is flat at 16.5–17.5 MHz across a $9\times$ span of the knob. The reason is structural: each slot gathers across $NGROUP$ groups and there are $BP_C/NGROUP$ slots, so the mux volume is $NGROUP$ -invariant by construction and the crossbar is simultaneously the area floor and the critical path. Least squares gives $LUT = 615\,985 + 18\,415\,474/NGROUP$ within 5.7%. Banking, which replaces the flat register array plus crossbar with index-addressed banks, is therefore not an optimisation of a working parallel design but the thing that makes it implementable.

Within the banked family the plan had assumed cycles scale linearly with bank count. Cycle-measuring five geometries showed that they do not. The counts fit the exact closed form

$$\text{cycles} = 60(G_C + G_V) + 420 + (2G_V + G_C), \quad (1)$$

where G_C, G_V are the check and variable group counts and the constant $420 = 60 \cdot 7$ is a per-iteration pipeline tail (seven drain cycles in each of the $LEGS \cdot ITERS = 60$ iterations) that exists regardless of banking. Only the first term shrinks with banking, so $4\times$ banking ($16/48 \rightarrow 64/192$) yields $2.28\times$ fewer cycles, $2085 \rightarrow 913$, and the tail grows from 20% to 47% of the budget. The floor is the full-parallel geometry, $G_C = G_V = 1$: 543 cycles.

Table 4: Post-route implementation of two banked geometries on the VU47P (Vivado 2025.2, 5.0 ns target, default directives). The shipped KV260 row is the silicon measurement of §4.

Geometry	Cycles	LUTs	Util. (%)	F _{max} (MHz)	Latency (µs)
64/192	913	247 434	19.0	150.4	6.07
144/864	543	994 700	76.3	97.3	5.58
16/48, KV260	2085	—	—	133.3	15.64

This configuration could not be generated when the qualification was written; three generator defects (routing-network lanes sized by bank count rather than tap count, a zero-width row address at $\lceil \log_2 1 \rceil = 0$, and ROM literals beyond Verilator’s 65 536-bit limit) were fixed in Task B0 Option A, after which 48/288, 144/432 and 144/864 all pass the 40-shot golden 40/40 at 789, 605 and 543 cycles, with the header for every previously qualified geometry byte-identical apart from two guard lines. Since the 420-cycle tail is LEGS · ITERS · 7, the dominant remaining latency lever once banking is exhausted is the iteration budget, ITERS, which is set by the decoder’s LER at the operating point rather than by the hardware.

A large FPGA: it fits, and the clock collapses

Both 64/192 and 144/864 were implemented end to end (synthesis, opt, place, phys_opt, route) on the AMD Virtex UltraScale+ HBM xcvu47p-fsvh2892-2-e of AWS’s F2 instances, with Vivado 2025.2 on a z1d.2xlarge build instance, at a 5.0 ns target, default directives, no floor-planning and no fanout constraints. The instance lived 7.3 hours and cost about \$5.50; 64/192 implemented in 65 minutes, 144/864 in 5 h 40 min. No FPGA instance was rented: the build only needs the Vivado Enterprise licence that the FPGA Developer AMI bundles, and a bitstream is never loaded. Table 4 gives the post-route result.

The fit gate passes: 76.3 % is under the 90 % criterion, BRAM and LUTRAM are at zero and registers at 9.4 %, so LUTs are the only resource that matters. The latency case for full-parallel on FPGA does not pass: $1.68\times$ fewer cycles at a $1.55\times$ slower clock is a net $1.09\times$ for $4.0\times$ the area. 64/192 delivers 92 % of the latency in a quarter of the device. The clock loss is not logic depth. The worst path has eight logic levels and a 10.251 ns datapath delay of which 0.750 ns (7.3 %) is logic and 9.501 ns (92.7 %) is routing: a control counter, `pc_reg[1]`, reaching a min-sum cell across a die large enough to need three super logic regions, with Laguna SLR-crossing tiles inside the congested windows. It is the fanout-7941 control net the out-of-context probe had flagged, and every standard mitigation (`MAX_FANOUT`, per-bank replication, SLR-aware floorplanning) is untried, so 97.3 MHz is a floor for

Table 5: ASAP7 7 nm predictive, post-route, same flow and knobs for both geometries. F_{max} is `report_clock_min_period`, setup-only, on a netlist whose hold violations are unrepaired (§6). Power is from a gate-level VCD of the 16/48 netlist at the 1 GHz SDC; it was not measured at 144/864.

	16/48	144/864
Cycles (worst case)	2085	543
Design area at 50 % util. (µm ²)	81 090	434 450
Die (mm ²)	0.163	0.869
F _{max} , setup-only (MHz)	686.13	614.59
WNS at 1000 ps (ps)	−622.73	−1117.97
Latency (µs)	3.04	0.88
DRC violations	0	0
Hold violations (unrepaired)	44 704	35 616
Power, steady BP (W)	0.149	—
Power, idle, clock running (W)	0.138	—
Energy per 2085-cycle window (µJ)	0.31	—

this design rather than its ceiling. Two calibration lessons are worth carrying forward. The same RTL synthesised to 803 518 CLB LUTs on the KV260 part with Vivado 2024.2 and to 1 000 035 on the VU47P with 2025.2, 24 % more, so cross-part LUT extrapolation is worth about $\pm 25\%$; and post-route utilisation moved less than 1 % off post-synthesis on this part. Sub-microsecond needs 543 MHz on 543 cycles; nothing in this family on any FPGA is within a factor of three of that.

7 nm predictive: the geometry penalty is mostly an FPGA artefact

The FPGA result raised a specific risk: the only ASIC-node clock this project had measured, 686.13 MHz, came from the 16/48 geometry, and the program had been quoting 543 cycles against it as if the clock were geometry-independent. Both geometries were therefore run through OpenROAD-flow-scripts [8] on the ASAP7 predictive 7 nm platform [9], with the Q7-08 latch register file, identical knobs (`CORE_UTILIZATION=45`, `PLACE_DENSITY=0.55`, SDC period 1000 ps, timing repair skipped because `repair_timing` segfaults on this netlist class), and RC-extracted post-route timing. The 144/864 run took about 20 hours on an EPYC box and completed with $rc = 0$. Table 5 summarises both.

The clock is geometry-dependent on the ASIC too, but only mildly: the step from the small to the full-parallel geometry costs -10.4% on ASAP7 against -35.3% on the VU47P. An ASIC has neither fixed routing tracks nor SLR boundaries, and its buffer trees absorb the high-fanout control net that dominated the FPGA path; roughly a third of the penalty transferred. Area scales sub-linearly, $5.36\times$ for $8.5\times$ the logic, where the FPGA paid $8.53\times$ in CLB LUTs; of the placer’s $+11.18\%$ over the raw netlist, routability inflation was

+1.28 % and timing-driven growth +9.90 %. The headline, 543 cycles/614.59 MHz = 0.88 μ s, is a sub-microsecond worst-case window on a 7 nm predictive node, DRC-clean, at 0.869 mm². Figure 3 places it against the FPGA points.

We state the two qualifications here rather than only in §6. First, every $F_{\max}$ in Table 5 is a *setup-only* figure: hold was never repaired, the 16/48 netlist carries 44 704 hold violations (worst −745.9 ps, 98 % on the D pins of the latch register file) and the 144/864 netlist 35 616 (worst −1486.9 ps). Hold does not scale with the clock period, so these netlists would not decode correctly at any frequency; the first gate-level co-simulation of the 16/48 netlist fails 40/40 in exactly the way STA predicts, and 686 MHz is the speed of the datapath, not of a working part. A separately repaired but unrouted run gave 527.66 MHz, so the defensible 16/48 band is 528–686 MHz. Second, ASAP7 is a proxy: the margin below 1 μ s is 12 %, a 1.2 $\times$ node penalty gives 1.06 μ s, and nothing in this work measures the gap to a production node.

Power with real switching activity

The default-activity power ORFS reports, 0.656 W, is not a decoder number. The routed 16/48 netlist (639 322 cells, among them 55 703 latches and 43 025 flops) was compiled with Verilator against the ASAP7 behavioural cell models and driven by the same testbench and 40 golden vectors the RTL passes; two 300-cycle VCD windows, one inside a steady-state BP iteration and one with the decoder parked, were fed to OpenSTA with the extracted SPEF, annotating 2 128 987 pin activities with none left unannotated. At the 1 GHz SDC the core draws 0.149 W during BP and 0.138 W idle; the default activity was 4.4 $\times$ too high, almost entirely in the combinational group (0.466 W $\rightarrow$ 0.0115 W). Ninety-three per cent of the real power is clock tree plus sequential internal: the ASAP7 platform marks integrated clock-gating cells DONT_USE, so every register toggles its clock pin every cycle and the datapath itself is about 10 mW. Energy per worst-case window is 149 pJ per cycle $\times$ 2085 cycles = 0.31 μ J, clock-independent to first order and inside the 1 μ J budget with 3 $\times$ margin, at 7 nm predictive; the activity comes from the not-quite-bit-exact simulation of the previous paragraph, and absolute watts on a predictive NLDM library should be read as ± 30 %.

6 Limitations and negative results

We list these with the same weight as the positive results; several redirected the design.

Hold on the latch register file. The routed 16/48 ASAP7 netlist has 44 704 hold violations (worst −745.9 ps), 43 802 of them on the D pins of the latch register file: the latch write pulse is derived from `we &`

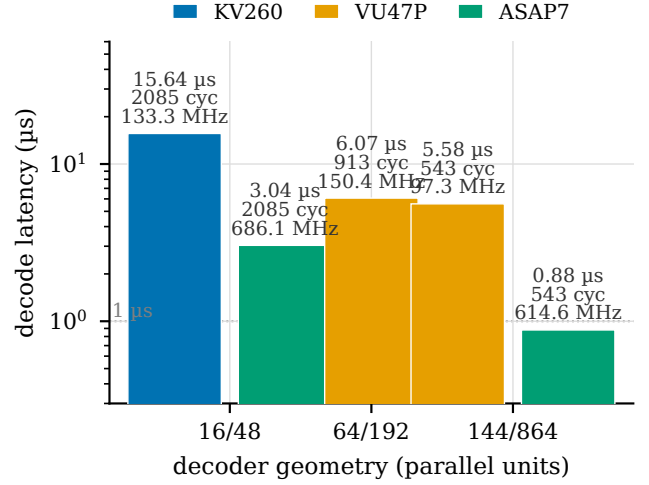


Figure 3: Worst-case decode latency against banking geometry on the three platforms measured: KV260 silicon (16/48), VU47P post-route (64/192, 144/864) and ASAP7 7 nm predictive post-route (16/48, 144/864). ASAP7 points are setup-only clocks on hold-unrepaired netlists. Data as in Tables 4 and 5.

`~clk` and realised as a clock-tree sub-network with 1.37 ns insertion delay against 0.47 ns for the flip-flops, so the latches close about 0.9 ns after the flops launch new data. The first gate-level co-simulation of the netlist reproduces the consequence: the 2085-cycle schedule runs exactly on all 40 golden shots, but 8–15 of 877 output fields differ and the decode does not converge. Hold is independent of clock period, and `repair_timing -hold` would need on the order of 10^6 delay buffers (DPL-0038); the fix is an integrated clock-gating cell or a re-timed write pulse, which the ASAP7 platform’s DONT_USE ICG* setting precluded here. Until it is made, every frequency in §5 is a setup-only datapath figure, and the same un-gated clock is why 93 % of the measured power is register clocking. The 144/864 netlist carries the same defect (35 616 violations, worst −1486.9 ps).

7 nm predictive is not a target node. ASAP7 [9] is a research kit. The margin between 0.88 μ s and the microsecond is 12 %; a 1.2 $\times$ node penalty at 28 nm would erase it. Nothing in this work measures that gap, and a 28 nm PDK is a gate we have not passed.

The streaming core does not fit. A windowed streaming variant with a runtime-addressed gather reached 162 % of the KV260’s LUTs and 113 % of its BRAM by estimate and exhausted 123 GB of host memory in technology mapping. It is bit-exact in simulation only; the fixed-permutation gather of §3 is the answer to it, not a refinement of it.

No OSD tail; no NGROUP. An OSD-0 post-processor did not reduce the logical error rate at the operating points measured, and the order-12 sweep that would have no place in a microsecond budget. A grouped partial-unroll knob (NGROUP) has no setting that both fits and is faster than the banked core.

Uncovered corners. Runt frames (fewer slices than the window) take a zero-pad path that no co-simulation vector exercises; the 64/192 and 144/864 geometries have been cycle-counted and synthesised but not run on hardware we own; and the gate-level co-simulation used behavioural replacements for two sequential cells because the vendor UDP models cannot be compiled by Verilator (an event-driven arbitration with the vendor models was attempted and remained X-contaminated).

One author, one lab. All measurements were made by the author on the author’s hardware. The deployment procedure has been executed on three wiped boards, but not yet by a third party; the repository invites and documents exactly that.

7 Reproducibility and availability

The repository [3] (Rust crates under MIT, `hw/` under Apache-2.0 for its patent grant) contains the golden model, every RTL variant in this paper, the Verilator co-simulation gates (run in CI on every pull request), the OpenROAD configurations, and the reports the numbers are copied from. `hw/product/deploy.sh` fetches the released KV260 bitstream and driver, programs a board over SSH, and runs the 40-shot golden on silicon; it has been run end to end on three wiped boards. A support policy (`hw/product/SUPPORT.md`) states what is supported and how to report a deployment.

The campaign data behind every table and figure — the message-width and schedule sweeps, the silicon LER campaigns and the ASAP7 timing and power reports — and the appliance-v1 bitstream are archived on Zenodo under DOI 10.5281/zenodo.XXXXXXX. Appendix A lists, for every number in the text, the file and section it was taken from.

References

- [1] Sergey Bravyi, Andrew W Cross, Jay M Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J Yoder. High-threshold and low-overhead fault-tolerant quantum memory. *Nature*, 627:778–782, 2024. doi: 10.1038/s41586-024-07107-7.
- [2] Tristan Müller et al. Improved belief propagation is sufficient for real-time decoding of quantum memory. *arXiv:2506.01779*, 2025.
- [3] Ruslan Malymon. aleph: quantum circuit simulator and QEC decoder hardware. <https://github.com/aleph-sim/aleph>, 2026.
- [4] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021. doi: 10.22331/q-2021-07-06-497.
- [5] Pavel Panteleev and Gleb Kalachev. Degenerate quantum LDPC codes with good finite length performance. *Quantum*, 5:585, 2021. doi: 10.22331/q-2021-11-22-585.
- [6] Abraham Waksman. A permutation network. *Journal of the ACM*, 15(1):159–163, 1968. doi: 10.1145/321439.321449.
- [7] Bruno Beauquier and Eric Darrot. On arbitrary size Waksman networks and their vulnerability. *Parallel Processing Letters*, 12(3–4):287–296, 2002. doi: 10.1142/S0129626402001008.
- [8] The OpenROAD Project. OpenROAD-flow-scripts. <https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts>, 2026.
- [9] Lawrence T Clark et al. ASAP7: A 7-nm FinFET predictive process design kit. *Microelectronics Journal*, 53:105–115, 2016. doi: 10.1016/j.mejo.2016.04.006.

A Provenance of every number

Each numeric claim in the text carries, in the L^AT_EX source, a comment naming the repository file and heading it was copied from. This table lists those files per section (headings abbreviated); the reports themselves are in **docs/** of the repository [3] at the commit tagged **preprint-v1**; the section headings quoted in each comment locate the exact table or paragraph.

Section	Source files (docs/ unless noted)
§2 The decoder	perf/qec-q5-gross.md; perf/qec-q5-circuit-dem.md; perf/qec-q7-fixed-bp.md; perf/qec-q5-bposd.md; perf/data/qec-q7-fixed-bp.csv; perf/data/qec-q7-budget.csv
§3 Hardware architecture	perf/qec-q7-fixed-bp.md; superpowers/specs/2026-07-15-m9c-serial-gather-design.md; superpowers/specs/2026-07-16-m9c-aswaksmen-rightsize-design.md; qec/asic-architecture.md; qec/regfile-plan.md; qec/q7-06-ac1-batched-dma.md; perf/q7-02-asap7-timing.md
§4 Results on FPGA silicon	perf/qec-q7-fixed-bp.md; hw/product/README.md; hw/product/interface-spec.md; qec/q7-06-ac1-batched-dma.md; qec/q7-07-nonconvergence-policy.md; superpowers/specs/2026-07-25-q7-07-nonconvergence-policy-design.md
§5 Scaling and predictive ASIC	perf/q7-02-ngroup-sweep.md; qec/asic-architecture.md; qec/open-silicon-program.md; perf/q7-02-fullparallel-fpga.md; qec/b2-aws-build-runbook.md; perf/q7-02-b3-asap7-fullparallel.md; perf/q7-02-asap7-timing.md
§6 Limitations	perf/q7-02-asap7-timing.md; qec/open-silicon-program.md; perf/q7-02-b3-asap7-fullparallel.md; perf/qec-q7-fixed-bp.md; perf/q7-02-ngroup-sweep.md
§7 Reproducibility	hw/product/README.md